

SQL VISTAS (VIEWS): EL CONCEPTO AVANZADO

Una **guía visual** sobre la abstracción y la eficiencia en bases de datos.

¿QUÉ ES UNA VISTA?



Una **vista (view)** en SQL es una tabla virtual basada en el resultado de una consulta **SELECT**.

No almacena datos físicamente, sino que muestra datos de una o más tablas.

Cuando se consulta la vista, se ejecuta la definición definida en ella.

1. ¿PARA QUÉ SIRVE?



- ✓ Simplificar consultas complejas.
- ✓ Mejorar la seguridad limitando el acceso a ciertas columnas o filas.
- ✓ Proporcionar una interfaz consistente.
- ✓ Aumentar el nivel de abstracción de datos reportes y aplicaciones.
- ✓ Facilitar el mantenimiento de las consultas.



Unión de Tablas

2. SINTAXIS: CREATE VIEW



```
CREATE VIEW nombre_vista AS
SELECT columna1, columna2, ...
FROM tabla_o_vista
WHERE condición;
```

- **CREATE VIEW:** crea la vista.
- **nombre_vista:** nombre que tendrá la vista.
- **SELECT:** define las columnas y los datos que mostrará la vista.
- **FROM:** indica las tablas base.
- **WHERE (optional):** filtra las filas.

3. EJEMPLO PRÁCTICO

Supongamos que tenemos las siguientes tablas:

Tabla: Empleados

id	nombre	cargo	salario	depto_id
1	Ana Pérez	Analista	3500	1
2	Luis Gómez	Desarrollador	4200	1
3	María Ruiz	Diseador	4200	2
4	Carlos Díaz	Analista	3300	2
4	Carlos Díaz	Analista	3200	2
5	Carlos Díaz	Analista	3300	1
6	Juan Torres	Gerente	4000	1

Tabla: Departamentos

id	departamentos
1	TI
2	Marketing

Queremos crear una vista que muestre nombre, su cargo y el nombre del departamento, su cargo.

Código SQL:

```
CREATE VIEW vista_empleados AS
SELECT e.nombre, e.cargo, d.departamento
FROM Empleados e
JOIN Departamentos d
ON e.depto_id = d.id;
```

Consulta a la vista:

```
SELECT * FROM vista_empleados;
```



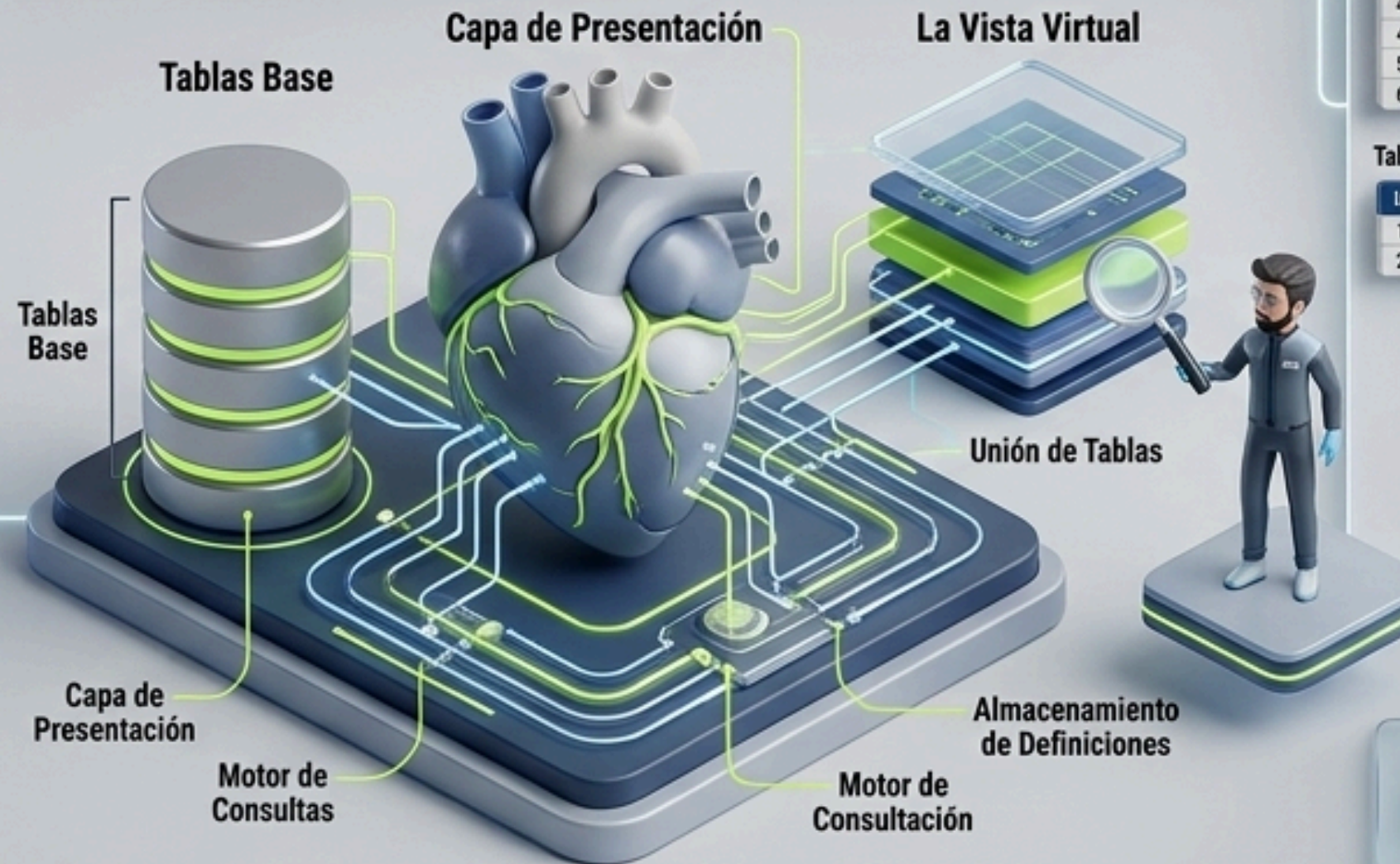
Unión de Tablas

Resultado:

nombre	cargo	departamento
Ana Pérez	Analista	TI
Luis Gómez	Desarrollador	TI
María Ruiz	Diseador	Marketing
Carlos Díaz	Analista	Marketing
Juan Torres	Gerente	TI

Ejemplos de uso:

- `SELECT * FROM vista_empleados WHERE 'Analista';`
- `SELECT COUNT(*) FROM vista_empleados;`
- `JOIN` con cierta tabla a tablas.



4. VENTAJAS DE USAR VISTAS



EN RESUMEN Las vistas mejoran la seguridad, simplifican las consultas y mejoran la consistencia de los datos.



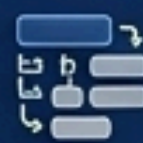
Seguridad
Control de Acceso Granular



Rendimiento
Consultas Pre-calculadas vs Ad-hoc



Abstracción
Capa Lógica



Simplicidad
Consultas Simplificadas



Comparación
Vista vs Tabla Física

SQL FUNCIONES (FUNCTIONS): EL PODER DEL CÓDIGO REUTILIZABLE

Una **guía visual** sobre la encapsulación y la manipulación de datos en bases de datos.

¿QUÉ ES UNA FUNCIÓN?



Una función SQL es un bloque de código reutilizable que acepta parámetros, realiza una tarea y devuelve un valor.

Se puede invocar directamente en consultas SELECT.

1. ¿PARA QUÉ SIRVE?



- **Encapsular lógica:** Oculta cálculos complejos.
- **Reutilizar código:** Úsala en múltiples consultas.
- **Simplificar consultas:** Hazlas más legibles y manejables.
- **Asegurar consistencia:** Uniforma la manipulación de datos.
- **Mejorar mantenimiento:** Modifica la lógica en un solo lugar.



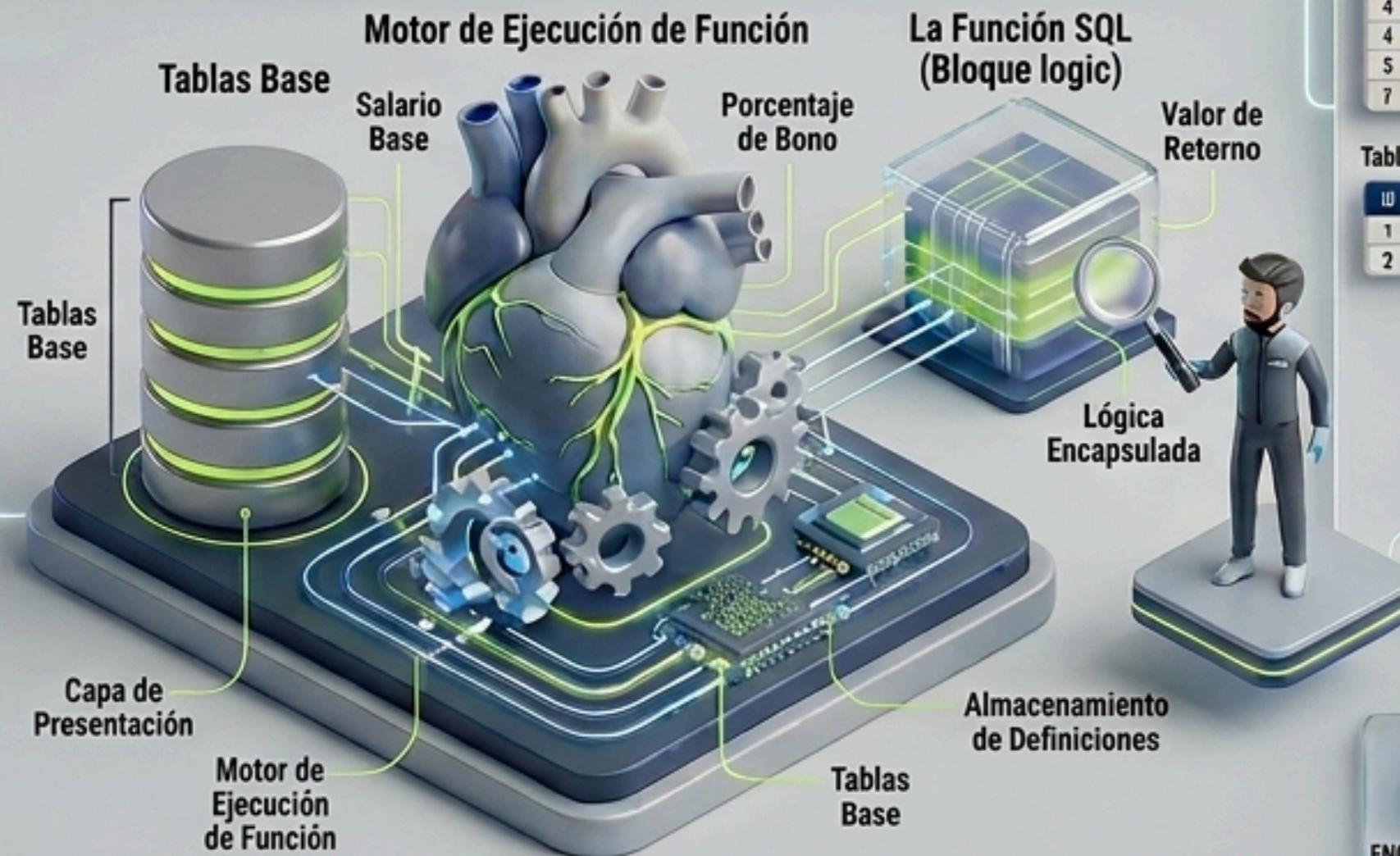
Unión de Tablas

2. SINTAXIS: CREATE FUNCTION



```
CREATE FUNCTION calcula_bonificacion (NOMBRE, VENTA,
PCTBONO)
RETURNS DECIMAL(10,2)
BEGIN
    RETURN (venta * pctbono) / 100;
END;
```

- **CREATE FUNCTION:** crea la función
- **nombre:** Nombre: nombre que tendrá
- **parametros:** lista de parámetros con sus tipos.
- **RETURNS TIPO:** tipo de dato devuelto.
- **BEGIN.. END:** Bloques de código
- **RETURN:** valor que devuelve.



3. EJEMPLO PRÁCTICO

Supongamos que tenemos las siguientes tablas:

Tabla: Empleados

ID	nombre	cargo	salario	departa
1	Adrián	Analista	4500	1
2	Lola Gómez	Desarrollador	4200	1
3	Miguel Ruiz	Analista	4260	2
4	Carlos Ruiz	Analista	3500	2
4	Carlos Ruiz	Analista	2200	2
5	Concepción	Analista	5300	1
7	Juan Torres	Analista	4000	1

Tabla: Departamentos

ID	departamento
1	IT
2	Marketing

Queremos crear una vista que muestre el nombre del empleado, su cargo y el nombre del departamento, en una sola consulta.

Código SQL (Creación de Función)

```
CREATE VIEW vista_empleados AS
SELECT nombre, cargo, departamento
FROM empleados
JOIN departamentos
ON empleados.departa = departamentos.ID;
```

Consulta a la vista:

```
SELECT nombre, cargo, departamento
FROM vista_empleados;
```

Unión de Tablas

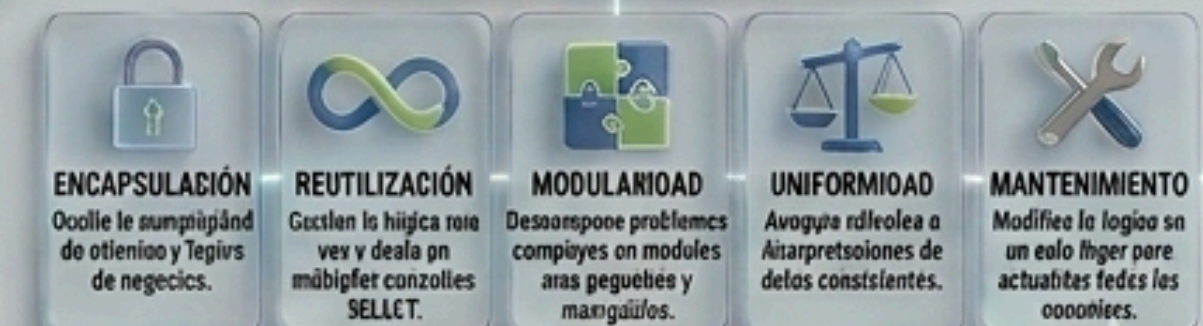
Resultado:

nombre	cargo	departamento
Adrián	Analista	IT
Lola Gómez	Desarrollador	IT
Miguel Ruiz	Analista	Marketing
Carlos Ruiz	Analista	Marketing
Concepción	Analista	IT
Juan Torres	Analista	IT

Ejemplos de uso:

- **SELECT** nombre, cargo, departamento FROM vista_empleados WHERE departamento = 'IT';
- **SELECT COUNT(*)** FROM vista_empleados;
- **JOIN** con otras tablas o vistas

4. VENTAJAS DE USAR FUNCIONES



EN RESUMEN

Las funciones SQL, a través de la encapsulación, reutilización y modularidad, facilitan operaciones complejas y mejoran la legibilidad y el mantenimiento del código.



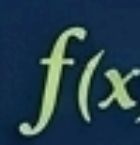
Encapsulación
Oculta la complejidad lógica



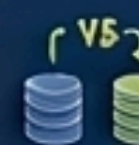
Modularidad
Descompone código complejo



Reutilización
Ahorro de tiempo y espacio



Cálculos
Operaciones f(x) personalizadas



Comparación
Función vs Consulta Ad-hoc

SQL ESQUEMAS (SCHEMAS): LA ESTRUCTURA FUNDAMENTAL

Una guía visual sobre la organización y la jerarquía en bases de datos.

¿QUÉ ES UN ESQUEMA?



Un esquema (schema) es un contenedor lógico de objetos de base de datos.

Proporciona un namespace que organiza tablas, vistas, y otros objetos de manera estructurada.

No contiene datos en sí mismo, sino que define la estructura.

1. ¿PARA QUÉ SIRVE?



- ✓ Simplificar consultas complejas.
- ✓ Mejorar la seguridad limitando el acceso a ciertas columnas o filas.
- ✓ Proporcionar una interfaz consistente.
- ✓ Aumentar el nivel de abstracción de datos para reportes y aplicaciones.
- ✓ Facilitar el mantenimiento de las consultas.



Unión de Tablas

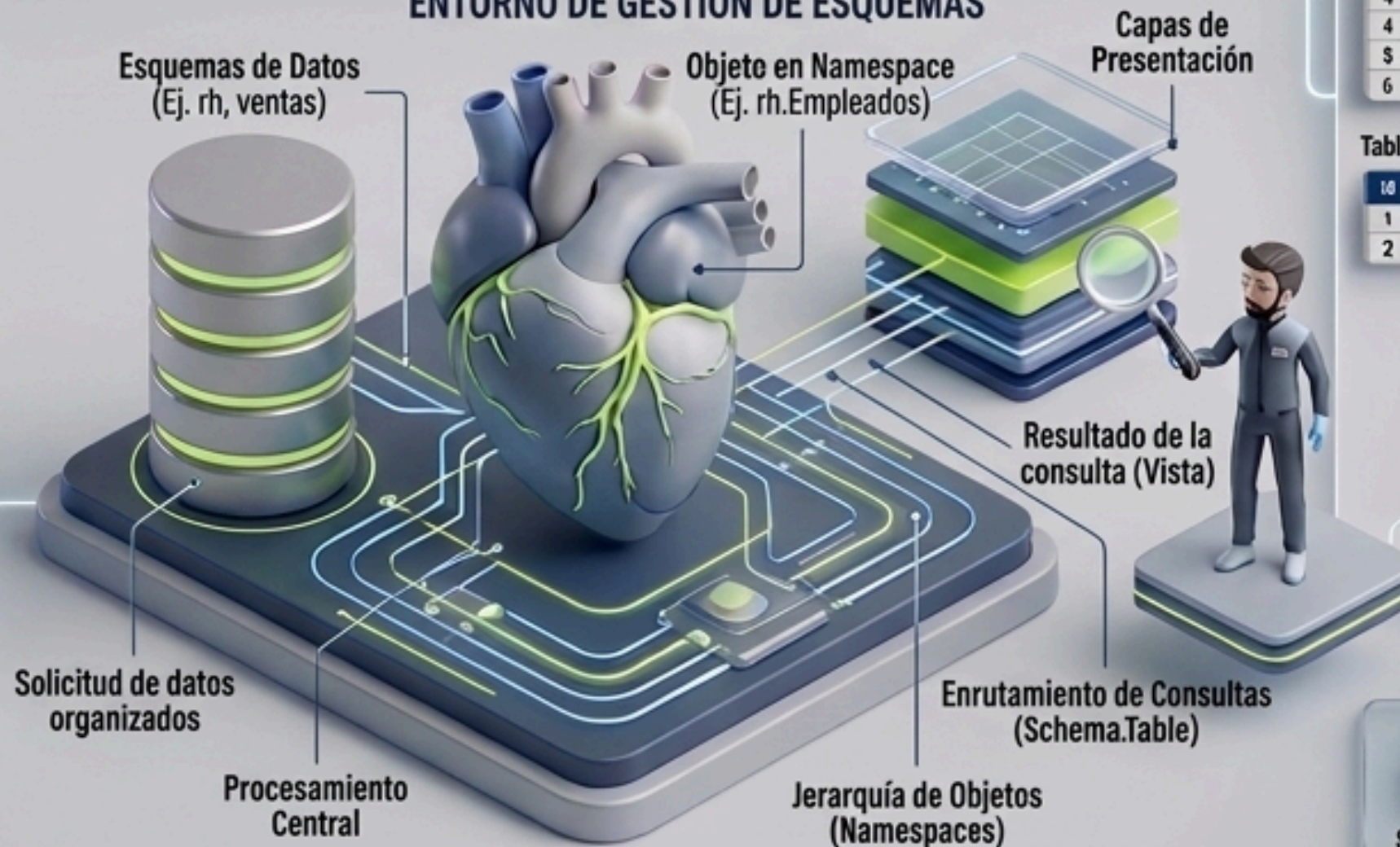
2. SINTAXIS: CREATE SCHEMA



```
CREATE SCHEMA `nombre_esquema`
[AUTHORIZATION `propietario`]
[objects...];
```

- **CREATE SCHEMA:** crea el contenedor lógico.
- **nombre_esquema:** nombre único del namespace.
- **AUTHORIZATION:** (opcional) define al dueño.
- **objects:** (opcional) define objetos de inmediato.

ENTORNO DE GESTIÓN DE ESQUEMAS



3. EJEMPLO PRÁCTICO

Supongamos que tenemos las siguientes tablas:

Tabla: Departamento: departamentos

id	nombre	codigo	telefono	depto_id
1	And Prey	Analiela	3500	1
2	Lole Gómes	Desarrollador	4200	1
3	Marie Reiz	Dubaiador	4200	2
4	Carlos Ollas	Analiela	3300	2
4	Carloe Ditz	Analiela	3300	2
5	Carloe Dier	Analiela	3300	1
6	Joan Torres	Gerente	4000	1

Tabla: Empleados:

id	departamentos
1	RI
2	departamentos

Queremos crear una vista que muestre nodal clqatsts, se curgs y al manal del departamento, eu surge.

Código SQL: creación de esquemas y tablas

```
-- Consulta entre esquemas
SELECT e.nombre, d.departamentos
FROM rh.Empleados e
JOIN ventas.Departamentos d
ON e.depto_id = d.id;
```

Consulta a la vista:

```
SELECT * FROM vista_empleados;
```



Unión de Tablas

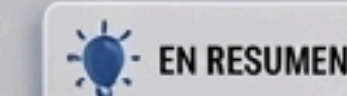
Resultado de la unión entre esquemas:

nombre	cargo	departamento
Ans Frey	Analiela	T1
Lole Gliben	Desarrollador	T1
Marie Rart	Creador	Marketing
Carloe Dros	Analiela	Marketing
Joan Temes	Gerente	T1

Ejemplos de uso:

- `SELECT * FROM rh.Empleados;`
- `CREATE TABLE ON rh.* TO 'vnaalista';`
- `DROP SCHEMA ventas CASCADE;`

4. VENTAJAS DE USAR ESQUEMAS



EN RESUMEN

Un esquema es una herramienta potente para organizar, asegurar y gestionar la complejidad de los objetos de tu base de datos, separándolos en contenedores lógicos.



Seguridad

Permisos Granulares por Esquema.



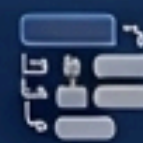
Organización

Jerarquia vs Almacenamiento Plano.



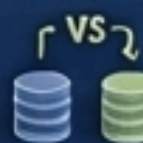
Abstracción

Namespaces vs Entidades Físicas.



Simplicidad

Gestión de Múltiples Esquemas (rh, contabilidad).



Base de Datos

BASE DE DATOS (Nivel Físico/Lógico, Contenedor Completo)

SQL ÍNDICES (INDEXES): EL CONCEPTO AVANZADO

Una **guía visual** sobre la optimización y la velocidad de búsqueda en bases de datos.

¿QUÉ ES UN ÍNDICE?



Un índice (index) en SQL es una estructura de datos virtual o física asociada a una tabla que acelera la recuperación de filas.

No duplica la tabla completa, sino que guarda punteros ordenados hacia los datos reales.

Cuando se consulta la tabla, el motor de base de datos busca en el índice en lugar de escanear toda la tabla (evita el Full Table Scan).

1. ¿PARA QUÉ SIRVE?



- ✓ Acelerar drásticamente las consultas con cláusulas WHERE, JOIN y ORDER BY.
- ✓ Evitar el escaneo completo de tablas con millones de registros.
- ✓ Garantizar la unicidad de los datos (mediante índices únicos).
- Mejorar el rendimiento general del servidor de base de datos.
- Optimizar el ordenamiento interno de los registros almacenados.



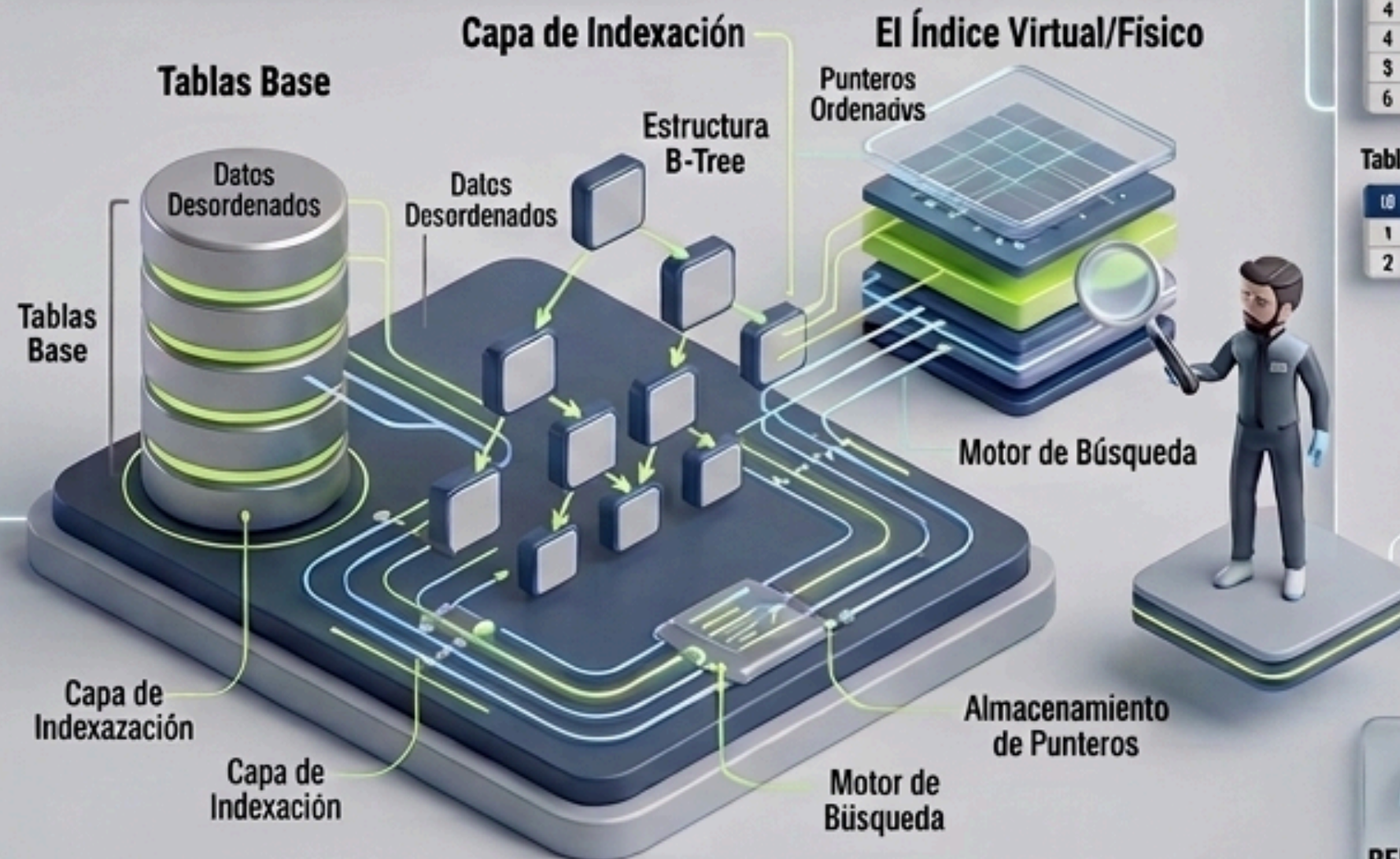
Unión de Tablas

2. SINTAXIS: CREATE INDEX



```
CREATE INDEX nombre_indice ON
tabla (columna1, columna2, ...);
WHERE condicion_parcial; --
(Opcional en algunos SGBD)
```

- **CREATE INDEX:** Comando que inicia la creación de la estructura.
- **nombre_indice:** Nombre que identificará el índice en el sistema.
- **tabla (columna1, columna2, ...):** Especifica sobre qué tabla y qué columnas se construye.
- **WHERE (opcional):** Crea un índice parcial (solo indexa filas que cumplen la condición).



3. EJEMPLO PRÁCTICO

Supongamos que tenemos las siguientes tablas:

Tabla: Empleados

ID	nombre	cargo	salario	depto_id
1	Andrés Pray	Analista	3300	1
2	Lola Gómez	Desarrollador	4200	1
3	Maria Ruiz	Desarrollador	4200	2
4	Carlos Gbcs	Analista	3300	2
4	Carlos Ortiz	Analista	3300	3
5	Carlos Olier	Analista	3500	1
6	Juan Terves	Gerente	4300	1

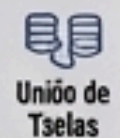
Queremos crear un índice en la tabla de empleados para acelerar las búsquedas por el campo cargo, ya que es la consulta más frecuente del sistema.

Código SQL:

```
CREATE INDEX idx_empleados_cargo ON
empleados (cargo);
```

Consulta a la tabla:

```
SELECT * FROM empleados
WHERE cargo = 'Analista';
```



Unión de
Tabelas

Tabla: Departamentos

ID	departamentos
1	TI
2	Marketing

Resultado (Búsqueda Instantánea sin recorrer toda la tabla):

nombre	cargo	departamento
Andrés Pray	Analista	TI
Lola Gómez	Desarrollador	TI
Maria Ruiz	Analista	Marketing
Juan Terves	Analista	TI

Ejemplos de uso:

- `SELECT * FROM empleados WHERE cargo = 'Desarrollador';`
- `JOIN` con cierta tabla a tablar.

4. VENTAJAS DE USAR ÍNDICES



EN RESUMEN

Los índices mejoran drásticamente la velocidad, optimizan el rendimiento al meter la búsqueda y preparan la base de datos a las consultas sin afectar la estructura física de los datos. (Ej. Abuso de los índices y el uso de índices).



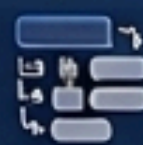
VELOCIDAD
Búsquedas en milisegundos.



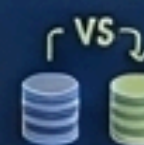
RENDIMIENTO
Consultas Indexadas vs Escaneo Completo.



OPTIMIZACIÓN
Estructuras en Arbol-B (B-Tree).



SIMPLICIDAD
Transparente para el usuario final.



COMPARACIÓN
Índice (Arbol) vs Tabla Física (Libro completo).

SQL PROCEDIMIENTOS ALMACENADOS: EL CONCEPTO AVANZADO

Una **guía visual** sobre la encapsulación y la lógica de negocio en bases de datos.

¿QUÉ ES UN PROCEDIMIENTO ALMACENADO?



Un procedimiento almacenado (Stored Procedure) es una colección de sentencias SQL compiladas y almacenadas en el servidor de base de datos. Se puede llamar por su nombre para ejecutar la lógica.

No solo consulta datos; puede contener lógica de control de flujo (IF, WHILE) y comandos DML (INSERT, UPDATE, DELETE). Al estar pre-compilado, puede ser más rápido que las consultas ad-hoc.

1. ¿PARA QUÉ SERVE?



- ✓ Simplificar complejas. (Simplifica tareas repetitivas y complejas).
- ✓ Mejorar el rendimiento. Al estar pre-compilados y optimizados por el servidor).
- ✓ Proporcionar seguridad. (Permite acceso a datos a través de lógica controlada, no directamente a tablas).
- ✓ Abstracción de lógica. (Mueve la lógica de negocio del cliente a la base de datos).
- ✓ Reutilización de código. Define la lógica una vez y úsala en múltiples aplicaciones).



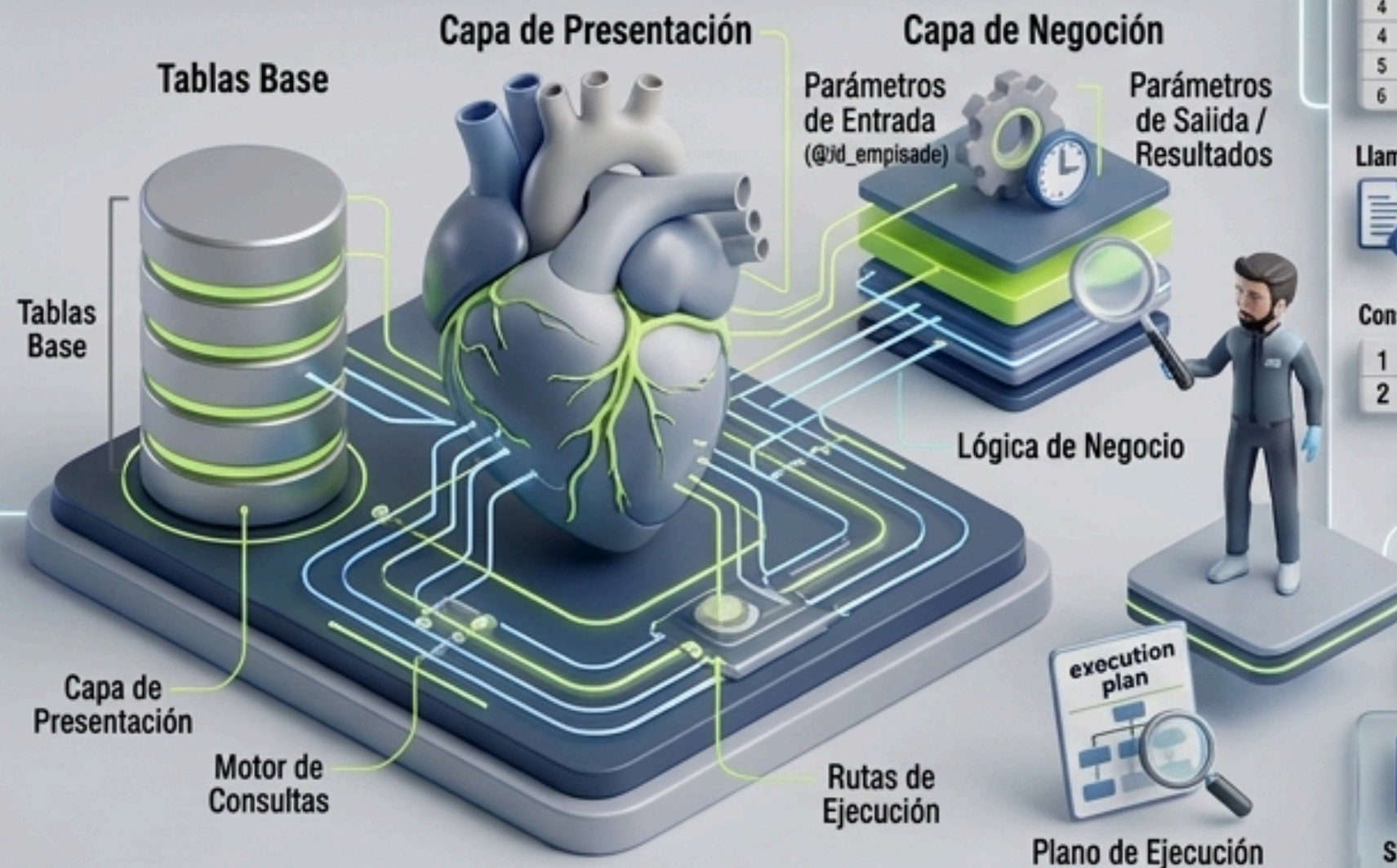
Unión de Tablas

2. SINTAXIS: CREATE VIEW



```
CREATE VIEW mostre_viste VS
SELECT columnal, columna2, ...
FROM tabla_o_viste
WHERE condición;
```

- CREATE VIEW: crea la vista.
- nombra_viste: nombre que tendrá la vista.
- SELECT: define las columnas y los datos que mostrará la vista.
- FROM: define las tablas bases.
- WHERE (optional): filtra los datos.



3. EJEMPLO PRÁCTICO

Supongamos que tenemos las siguientes tablas:

Tabla: Empleados

Id	nombre	cargo	salario	depts_id
1	Andrés	Analista	3300	1
2	Lola Gómez	Desarrollador	4200	1
3	María Niz	Desarrollador	4200	2
4	Carlos Ocas	Analista	3300	2
5	Carla Ortiz	Analista	3300	3
6	Carla Ortiz	Analista	3300	1
7	Joan Torres	Gerente	4000	1

Llamada al Procedimiento:

```
EXEC sp_CalcularSalarioPromedioTI
```

Consulta a la vista:

- 1 Unión
- 2 de Tablas



Queremos crear una vista que muestre el nombre del empleado, su cargo y el nombre del departamento, en un solo renglón.

Código SQL:

```
CREATE PROCEDURE sp_CalcularSalarioPromedio (
    @id_departamento INT,
    @salario_promedio DECIMAL(10,2) OUTPUT
)
AS
BEGIN
    SELECT @salario_promedio = AVG(salario)
    FROM Empleados
    WHERE depts_id = @id_departamento;
END
```

Llamada (T-SQL):

```
SELECT @avg_sal FROM (16,12);
EXEC sp_CalcularSalarioPromedio 1, @avg_sal OUTPUT;
SELECT @avg_sal;
```

Resultado:

1	3500.00
---	---------

Ejemplos de uso:

- EXEC sp_CrearNuevoEmpleado @nombre, @cargo, @salario
- EXEC sp_ActualizarSalarioPorDepartamento @id_dept, @porcentaje
- CALL sp_ObtenerDatosAntiguos(@fecha)

4. VENTAJAS DE USAR PROCEDIMIENTOS ALMACENADOS



SEGURIDAD

Control de acceso granular a lógicas, no a tablas base



RENDIMIENTO

Ejecución rápida gracias a la pre-compilación y caché



MODULARIDAD

Unidad lógica compleja en módulos reutilizables y mantenibles



REUTILIZACIÓN

Lógica centralizada disponible para múltiples aplicaciones



ABSTRACCIÓN

Oculta la estructura compleja de la DB a los usuarios/apps



Seguridad

Encapsulación de lógica, permisos de ejecución



Rendimiento

Pre-compilación, caché del plan de ejecución



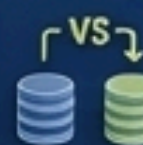
Abstracción

Lógica de negocio en la capa de datos



Mantenibilidad

Lógica centralizada en un solo lugar



Comparación

Procedimientos vs Consultas Ad-hoc



EN RESUMEN

Los beneficios de seguridad, rendimiento, reutilización y abstracción se logran al encapsular la lógica de negocio en procedimientos almacenados.

